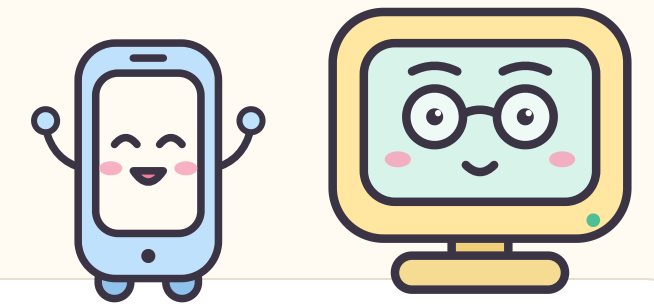




第 10 集 · 约 26 分钟

服务端③：状态与路由

一个请求进门，要过几道关

10

从零写出 LanShare

这一集你会学到

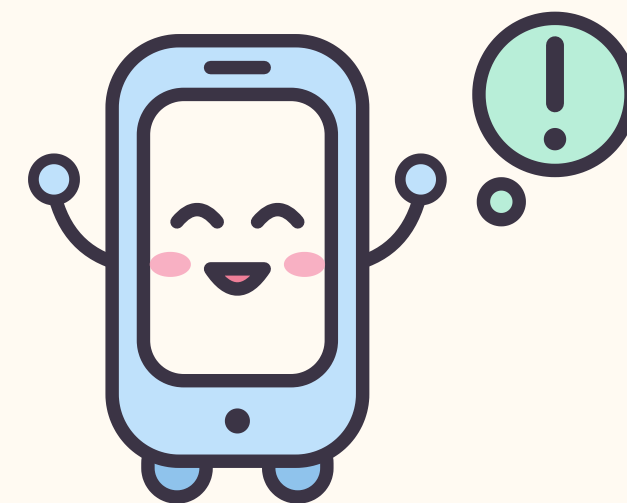
1. `async` 和 `tokio`：等网络的时候把线程让出来
2. `Config` 和 `AppState`：启动参数和运行中共享的状态，`Arc` 让大家共用一份
3. `trait` 和 `dyn`：桌面操作、日志由外面“插”进来
4. `axum` 的路由和中间件：Host 校验、安全响应头、Origin 检查
5. `macro_rules!` 和 `include_bytes!`：把网页编进 `exe`



先想一个问题

十部小手机同时连上来，老电脑需要开十个线程吗？

不需要。async 让线程在等网络的时候去照顾别的连接，几个线程就能服务成千上万个连接。





async: Python 和 Rust

Python • asyncio

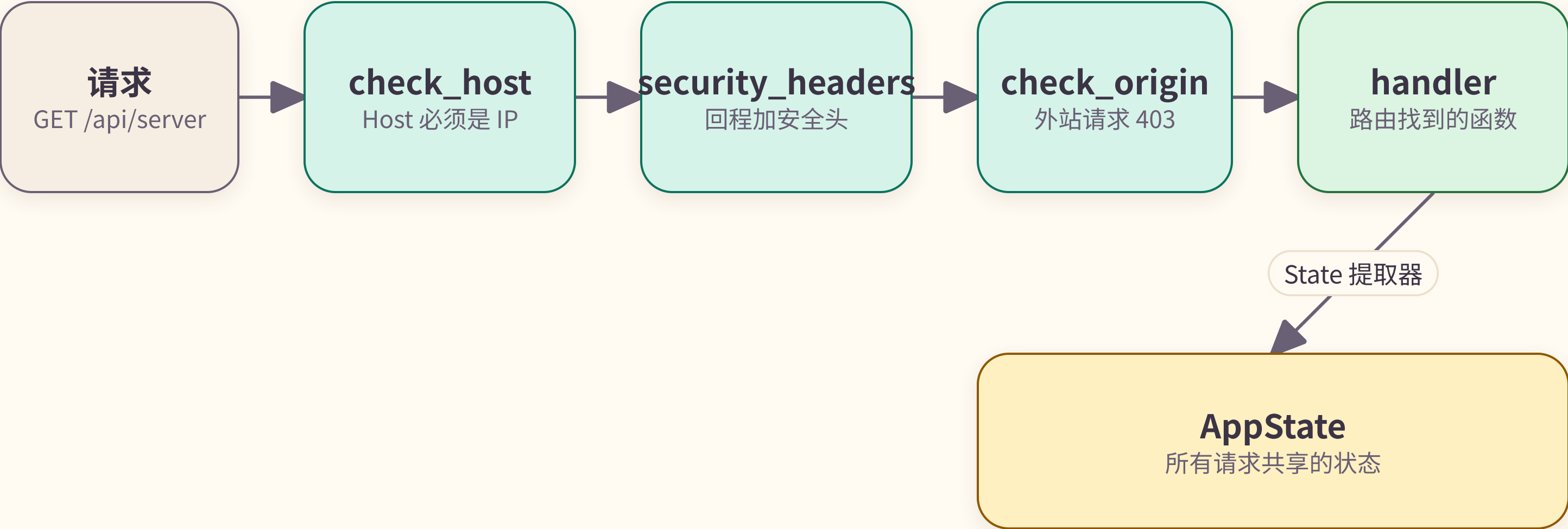
```
1 async def handle(reader, writer):
2     data = await reader.read(1024)    # 等网络：把线程让出去
3     writer.write(b"...")
4     await writer.drain()
5
6 asyncio.run(main())                  # 事件循环负责调度
```

Rust • tokio

```
1 async fn server_id(State(st): St) -> Response {
2     // 等网络的地方写 .await
3 }
4
5 let runtime = tokio::runtime::Runtime::new().unwrap();
6 runtime.block_on(async { /* 起服务 */ });
```



一个请求，三道关





加依赖：axum、tokio 和几个小工具

● ● ●

Cargo.toml

crates/lanshare/

第 7-26 行 · 共 26 行

```
7
8 [[bin]]
9 name = "LanShare"
10 path = "src/main.rs"
11
12 [dependencies]
13 axum = "0.8"
14 base64 = "0.22"
15 dirs = "6"
16 hex = "0.4"
17 if-addr = "0.13"
18 lanshare-proto = { path = "../lanshare-proto" }
19 rand_core = { version = "0.6", features = ["getrandom"] }
20 serde = { version = "1", features = ["derive"] }
21 serde_json = "1"
22 tokio = { version = "1", features = ["rt-multi-thread", "macros", "net", "fs", "io-util", "sync", "time", "signal"] }
23 windows-sys = { version = "0.61.2", features = ["Win32_System_SystemInformation", "Win32_Storage_FileSystem"] } 临时
24
25 [dev-dependencies]
26 tempfile = "3"
```



tokio 的八个特性开关

特性	提供什么	在哪儿用
rt-multi-thread	多线程运行时	本集跑服务
macros	select!、tokio::test 这些宏	下一集和测试
net	异步的 TcpListener、TcpStream	本集和下一集
time	sleep、timeout、计时器	下一集起
sync	信号量、一次性通道	读正文的名额、入口
io-util	read、write_all 这类读写方法	测试
signal	收 Ctrl+C	程序入口
fs	异步读写文件	仓库里的代码目前没用到：文件操作都交给了 spawn_blocking



server/mod.rs: 开头

● ● ●

mod.rs

crates/lanshare/src/server/

第 1-20 行 • 共 20 行

```
1  //! HTTP 服务：配置、共享状态、路由与中间件。
2  //!
3  //! 协议见 `docs/superpowers/specs/2026-10-02-lanshare-v2-rust-design.md`。
4
5  mod assets;
6
7  use std::net::{IpAddr, Ipv4Addr};
8  use std::path::{Path, PathBuf};
9  use std::sync::{Arc, Mutex};
10
11 use axum::Router;
12 use axum::extract::{Request, State};
13 use axum::http::{HeaderValue, StatusCode, header};
14 use axum::middleware::{self, Next};
15 use axum::response::{IntoResponse, Response};
16 use axum::routing::get; 临时
17 use base64::Engine;
18 use rand_core::{OsRng, RngCore};
19
20 use crate::storage::Storage;
```



从外面插进来的两样东西

mod.rs

crates/lanshare/src/server/

第 9-28 行 · 共 28 行

```

9 use std::sync::{Arc, Mutex};
10
11 use axum::Router;
12 use axum::extract::{Request, State};
13 use axum::http::{HeaderValue, StatusCode, header};
14 use axum::middleware::{self, Next};
15 use axum::response::{IntoResponse, Response};
16 use axum::routing::get; 临时
17 use base64::Engine;
18 use rand_core::{OsRng, RngCore};
19
20 use crate::storage::Storage;
21
22 /// 只给本机主界面用的桌面操作（在资源管理器里定位文件、打开共享文件夹）。
23 pub trait Desktop: Send + Sync {
24     fn reveal(&self, path: &Path);
25     fn open_folder(&self, path: &Path);
26 }
27
28 pub type Logger = Arc<dyn Fn(&str) + Send + Sync>;

```



Config: 启动参数

mod.rs

crates/lanshare/src/server/

第 33-52 行 • 共 52 行

```

33     /// 不给就随机生成（正常运行时都不给）。
34     pub key: Option<[u8; 32]>,
35     pub pin: Option<String>,
36     pub lan_ips: Vec<Ipv4Addr>,
37     pub desktop: Option<Arc<dyn Desktop>>,
38     pub log: Logger,
39 }
40
41 impl Config {
42     pub fn new(share_dir: impl Into<PathBuf>) -> Self {
43         Config {
44             share_dir: share_dir.into(),
45             key: None,
46             pin: None,
47             lan_ips: Vec::new(),
48             desktop: None,
49             log: Arc::new(|_| {}),
50         }
51     }
52 }

```



AppState: 运行中共享的状态

mod.rs crates/lanshare/src/server/

第 59-78 行 · 共 78 行

```

59     port: u16,
60     lan_ips: Vec<Ipv4Addr>,
61     pub(crate) storage: Storage,
62 }
63
64 pub(crate) fn random_hex(bytes: usize) -> String {
65     let mut buf = vec![0u8; bytes];
66     OsRng.fill_bytes(&mut buf);
67     hex::encode(buf)
68 }
69
70 /// 均匀分布的 6 位数字口令（拒绝采样，避免取模偏差）。
71 pub(crate) fn random_pin() -> String {
72     loop {
73         let n = OsRng.next_u32();
74         if n < u32::MAX - (u32::MAX % 1_000_000) {
75             return format!("{:06}", n % 1_000_000);
76         }
77     }
78 }

```



AppState: 创建和读取

mod.rs

crates/lanshare/src/server/

第 131-150 行 · 共 150 行

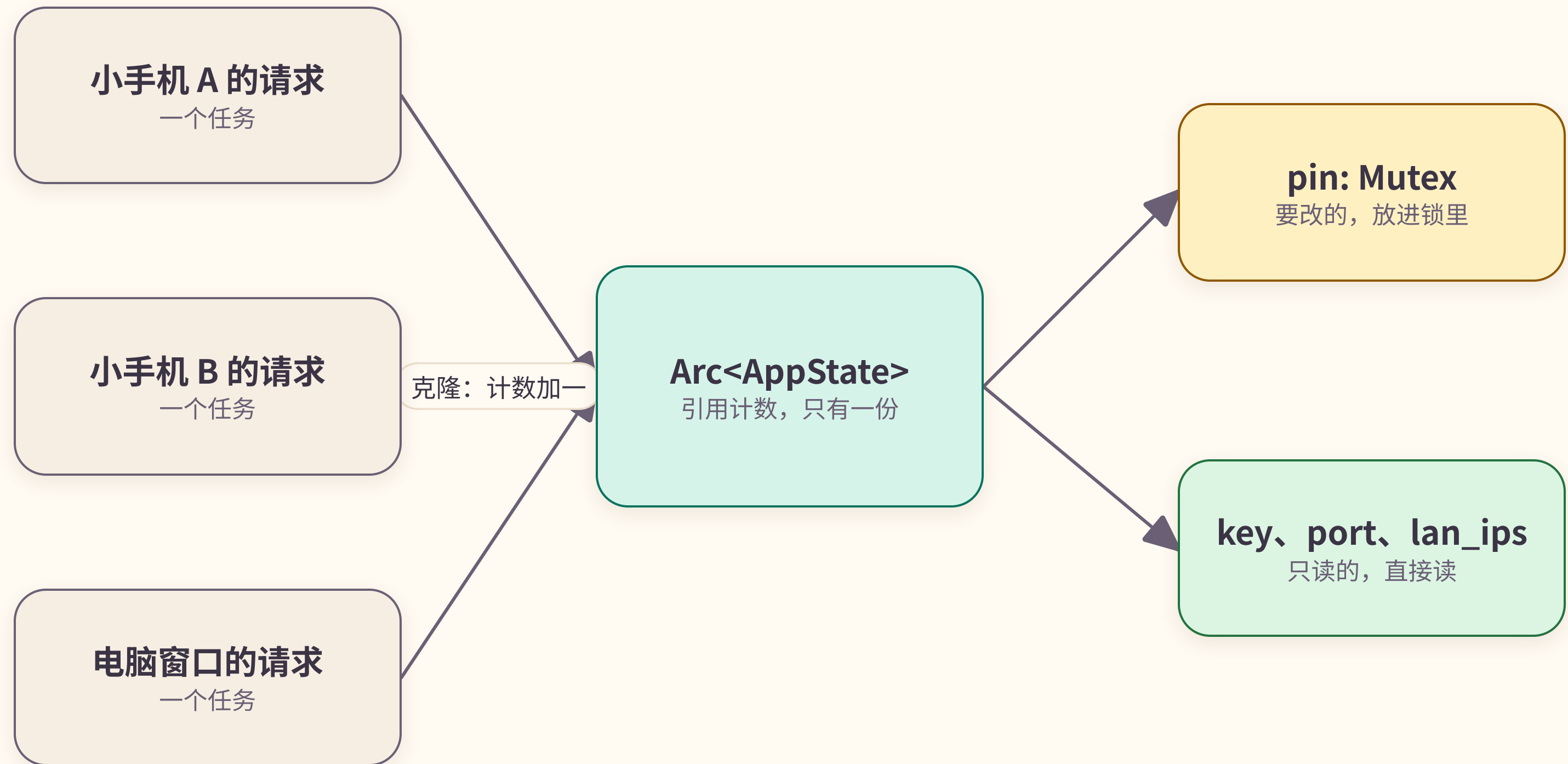
```

131
132     /// 启动时清理上次异常退出留下的临时文件，返回删除个数。
133     pub fn cleanup_stale_temp(&self) -> usize {
134         self.storage.cleanup_stale_temp()
135     }
136
137     pub(crate) fn key_fragment(&self) -> String {
138         base64::engine::general_purpose::URL_SAFE_NO_PAD.encode(self.key)
139     }
140
141     /// 二维码内容：主地址 + `#密钥`。`#` 后面的部分浏览器不会发到网络上。
142     pub fn entry_url(&self) -> String {
143         format!("{}", self.base_urls()[0], self.key_fragment())
144     }
145
146     /// 本机主界面的地址。
147     pub fn local_entry_url(&self) -> String {
148         format!("http://127.0.0.1:{}", self.port, self.key_fragment())
149     }
150 }

```

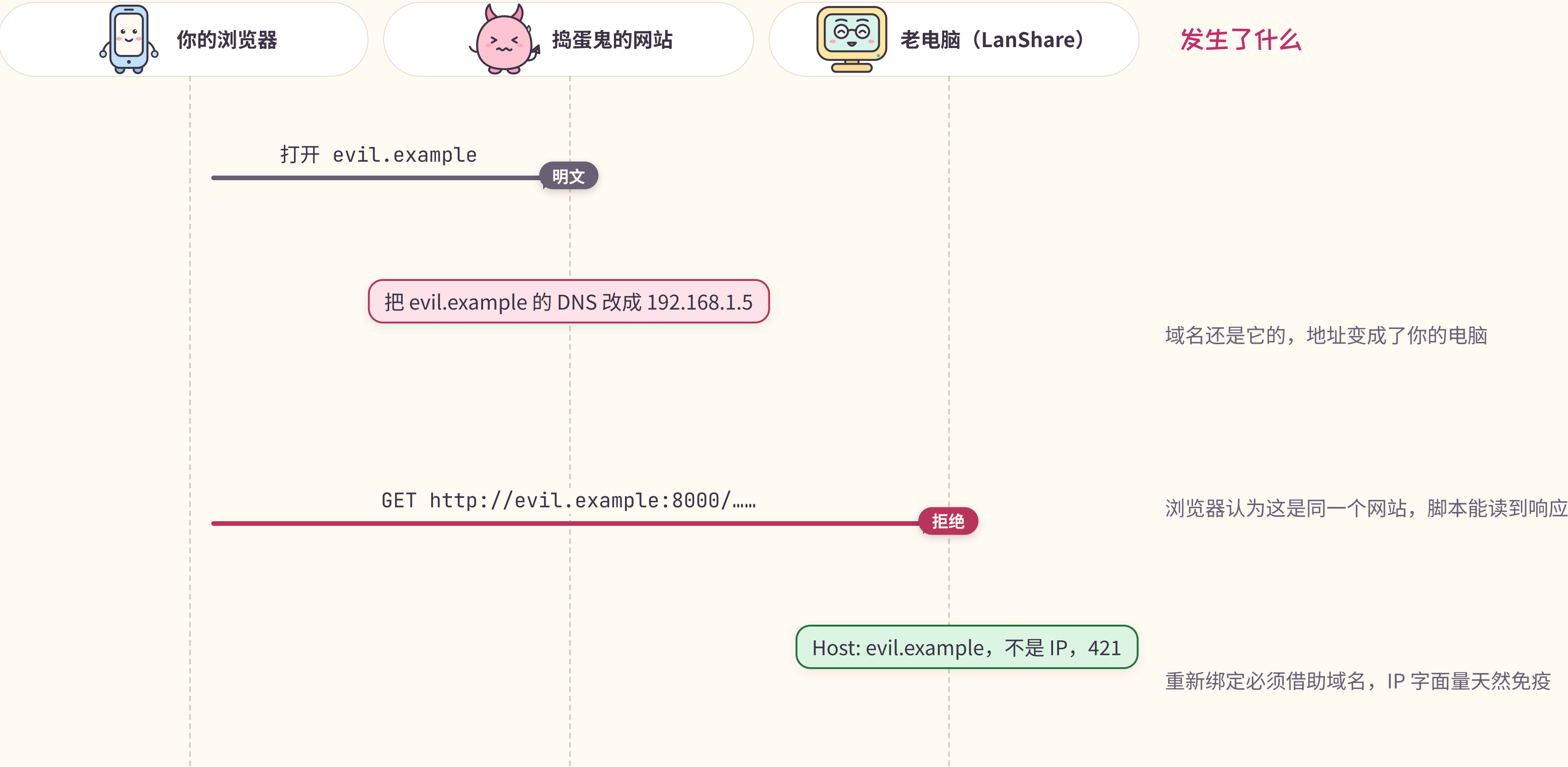


一份状态，大家共用





DNS rebinding: 借你的浏览器进门





第一道关：check_host

mod.rs crates/lanshare/src/server/

第 146–165 行 • 共 165 行

```

146     /// 本机主界面的地址。
147     pub fn local_entry_url(&self) -> String {
148         format!("http://127.0.0.1:{}", self.port, self.key_fragment())
149     }
150 }
151
152 /// 只接受 IP 字面量或 localhost 作为 Host: DNS rebinding 必须借助域名，IP 字面量天然免疫。
153 async fn check_host(request: Request, next: Next) -> Response {
154     let host = request.headers().get(header::HOST).and_then(|h| h.to_str().ok()).unwrap_or("");
155     let name = match host.rsplit_once(':') {
156         Some((name, port)) if !name.is_empty() && port.chars().all(|c| c.is_ascii_digit()) => name,
157         _ => host,
158     };
159     let name = name.trim_start_matches('[').trim_end_matches(']');
160     if name.eq_ignore_ascii_case("localhost") || name.parse:::<IpAddr>().is_ok() {
161         next.run(request).await
162     } else {
163         (StatusCode::MISDIRECTED_REQUEST, "unknown host").into_response()
164     }
165 }

```



第二道关：安全响应头

mod.rs crates/lanshare/src/server/

第 163–182 行 • 共 182 行

```

163         (StatusCode::MISDIRECTED_REQUEST, "unknown host").into_response()
164     }
165 }
166
167 async fn security_headers(request: Request, next: Next) -> Response {
168     let mut response = next.run(request).await;
169     let h = response.headers_mut();
170     h.insert(
171         header::CONTENT_SECURITY_POLICY,
172         HeaderValue::from_static(
173             "default-src 'none'; script-src 'self' 'wasm-unsafe-eval'; style-src 'self'; img-src 'self' blob:; \
174             connect-src 'self'; base-uri 'none'; form-action 'none'; frame-ancestors 'none'",
175         ),
176     );
177     h.insert(header::X_CONTENT_TYPE_OPTIONS, HeaderValue::from_static("nosniff"));
178     h.insert(header::REFERRER_POLICY, HeaderValue::from_static("no-referrer"));
179     h.insert(header::X_FRAME_OPTIONS, HeaderValue::from_static("DENY"));
180     h.insert(header::CACHE_CONTROL, HeaderValue::from_static("no-store"));
181     response
182 }
```



第一个 handler，和第三道关

● ● ●

mod.rs

crates/lanshare/src/server/

第 181-200 行 • 共 200 行

```

181     response
182 }
183
184 /// 公开：本次运行的标识（不是秘密），页面用它判断本地存的密钥是否属于当前这次运行。
185 async fn server_id(State(st): St) -> Response {
186     axum::Json(serde_json::json!({ "server_id": st.server_id(), "version": env!("CARGO_PKG_VERSION") })).into_response()
187 }
188
189 /// 带 Origin 头的请求（浏览器发的）必须来自本页面自己：挡住别的网站借用户的浏览器发请求（审查 M-2）。
190 /// 没有 Origin 的请求（非浏览器客户端、同源 GET）照常放行。
191 async fn check_origin(request: Request, next: Next) -> Response {
192     if let Some(origin) = request.headers().get(header::ORIGIN) {
193         let host = request.headers().get(header::HOST).and_then(|h| h.to_str().ok()).unwrap_or("");
194         let origin_host = origin.to_str().ok().and_then(|o| o.strip_prefix("http://")).unwrap_or("");
195         if host.is_empty() || !origin_host.eq_ignore_ascii_case(host) {
196             return (StatusCode::FORBIDDEN, "cross-origin request refused").into_response();
197         }
198     }
199     next.run(request).await
200 }

```



路由表

mod.rs crates/lanshare/src/server/

第 199-218 行 • 共 218 行

```

199     next.run(request).await
200 }
201
202 pub fn router(state: Arc<AppState>) -> Router {
203     Router::new()
204         .route("/", get(assets::index))
205         .route("/app.js", get(assets::app_js))
206         .route("/proto.js", get(assets::proto_js))
207         .route("/style.css", get(assets::style_css))
208         .route("/lanshare.wasm", get(assets::wasm))
209         .route("/favicon.ico", get(assets::favicon))
210         .route("/api/server", get(server_id))
211         .layer(middleware::from_fn(check_origin))
212         .layer(middleware::from_fn(security_headers))
213         .layer(middleware::from_fn(check_host))
214         .with_state(state)
215 }
216
217 /// 供 handler 使用的 State 类型别名。
218 pub(crate) type St = State<Arc<AppState>>;

```



assets.rs: 用宏生成六个 handler



assets.rs

crates/lanshare/src/server/

第 3-22 行 • 共 22 行

```
3 use axum::http::{HeaderValue, header};
4 use axum::response::{IntoResponse, Response};
5
6 macro_rules! asset {
7     ($name:ident, $file:literal, $mime:literal) => {
8         pub(crate) async fn $name() -> Response {
9             let body: &'static [u8] = include_bytes!(concat!(env!("CARGO_MANIFEST_DIR"), "/web/", $file));
10            let mut response = body.into_response();
11            response.headers_mut().insert(header::CONTENT_TYPE, HeaderValue::from_static($mime));
12            response
13        }
14    };
15 }
16
17 asset!(index, "index.html", "text/html; charset=utf-8");
18 asset!(app_js, "app.js", "text/javascript; charset=utf-8");
19 asset!(proto_js, "proto.js", "text/javascript; charset=utf-8");
20 asset!(style_css, "style.css", "text/css; charset=utf-8");
21 asset!(wasm, "lanshare.wasm", "application/wasm");
22 asset!(favicon, "icon.ico", "image/x-icon");
```



三个临时网页文件

index.html crates/lanshare/web/ 临时

第 1-6 行 · 共 6 行

```
1 <!doctype html>
2 <meta charset="utf-8">
3 <title>LanShare</title>
4 <link rel="stylesheet" href="style.css">
5 <p>临时页面：真正的界面在第 19 集。</p>
6 <script src="app.js"></script>
```



临时脚本

● ● ●

app.js crates/lanshare/web/ 临时

第 1-1 行 · 共 1 行

1

// 临时：第 19 集换成真正的界面



临时样式

● ● ●

style.css crates/lanshare/web/ 临时

第 1-1 行 · 共 1 行

1

/* 临时：第 19 集换成真正的样式 */



图标：从仓库拷贝





登记 server

● ● ●

lib.rs

crates/lanshare/src/

第 1-8 行 · 共 8 行

```
1  //! 局域网快传 v2：服务端与桌面外壳的库部分（`main.rs` 只负责启动）。
2
3  pub mod cli;
4  pub mod logging;
5  pub mod netinfo;
6  pub mod server;
7  pub mod storage;
8  pub mod textboard;
```



让它跑起来（临时）

main.rs crates/lanshare/src/ 临时

第 1-12 行 · 共 12 行

```
1 fn main() {
2     println!("LanShare 骨架跑起来了, 协议版本: {}", lanshare_proto::PROTOCOL);
3     // 临时: 先用 axum 自带的 serve 跑起来看看, 真正的入口在第 16 集 临时
4     let config = lanshare::server::Config::new(std::env::temp_dir().join("lanshare-demo")); 临时
5     let state = std::sync::Arc::new(lanshare::server::AppState::new(config, 8765)); 临时
6     println!("主界面地址: {}", state.local_entry_url()); 临时
7     let runtime = tokio::runtime::Runtime::new().unwrap(); 临时
8     runtime.block_on(async { 临时
9         let listener = tokio::net::TcpListener::bind("127.0.0.1:8765").await.unwrap(); 临时
10        axum::serve(listener, lanshare::server::router(state)).await.unwrap(); 临时
11    }); 临时
12 }
```



服务端骨架

```
lanshare/  
├── crates/  
│   └── lanshare/  
│       ├── src/  
│       │   ├── server/  
│       │   │   ├── assets.rs 这一集写的  
│       │   │   └── mod.rs 这一集写的  
│       │   ├── cli.rs  
│       │   ├── lib.rs 这一集写的  
│       │   ├── logging.rs  
│       │   ├── main.rs 这一集写的  
│       │   ├── netinfo.rs  
│       └── storage.rs
```

```
├── textboard.rs  
├── web/  
│   ├── app.js 这一集写的  
│   ├── icon.ico 这一集写的  
│   ├── index.html 这一集写的  
│   ├── lanshare.wasm 从仓库拷贝  
│   ├── proto.js  
│   └── style.css 这一集写的  
├── Cargo.toml 这一集写的  
├── lanshare-proto/ 13 个文件  
├── lanshare-wasm/ 2 个文件  
├── web-tests/ 2 个文件  
└── Cargo.toml
```



编译、跑测试

```
Windows PowerShell

PS D:\code\lanshare> cargo build
Updating crates.io index
Locking 38 packages to latest Rust 1.96.0 compatible versions
Adding atomic-waker v1.1.2
Adding axum v0.8.9
Adding axum-core v0.5.6
Adding base64 v0.22.1 (available: v0.23.1)
..... (省略 11 行, 都是 Adding)
Adding hyper v1.11.1
Adding hyper-util v0.1.21
..... (省略 14 行, 都是 Adding)
Adding tokio v1.53.1
Adding tokio-macros v2.7.2
Adding tower v0.5.3
..... (省略 4 行, 都是 Adding)
Compiling lanshare-proto v2.0.0 (D:\code\lanshare\crates\lanshare-proto)
Compiling lanshare v2.0.0 (D:\code\lanshare\crates\lanshare)
Compiling lanshare-wasm v2.0.0 (D:\code\lanshare\crates\lanshare-wasm)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 18.25s

PS D:\code\lanshare> cargo test -p lanshare --lib
```



运行，然后敲敲门

```
Windows PowerShell

PS D:\code\lanshare> cargo run -p lanshare
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.24s
    Running `target\debug\LanShare.exe`
LanShare 骨架跑起来了，协议版本: lanshare/2
主界面地址: http://127.0.0.1:8765/#pfBn2xpnu_0Bp4mxpFZMuCFhILytyMK4MI7f1wLTDcU

PS D:\code\lanshare> curl.exe -si http://127.0.0.1:8765/api/server
HTTP/1.1 200 OK
content-type: application/json
content-security-policy: default-src 'none'; script-src 'self' 'wasm-unsafe-eval'; style-src 'self'; img-src 'self'
blob:; connect-src 'self'; base-uri 'none'; form-action 'none'; frame-ancestors 'none'
x-content-type-options: nosniff
referrer-policy: no-referrer
x-frame-options: DENY
cache-control: no-store
content-length: 66
date: Fri, 02 Oct 2026 15:19:34 GMT

{"server_id":"06585ac0d7bea859549d9cfdd42f0e92","version":"2.0.0"}

PS D:\code\lanshare> curl.exe -si -H "Host: evil.example" http://127.0.0.1:8765/
HTTP/1.1 421 Misdirected Request
```



选择题

路由表里三个 layer 的书写顺序是 `check_origin`、`security_headers`、`check_host`。一个请求进来，最先执行的是哪个？

A `check_origin`

B `security_headers`

C `check_host`



选择题

路由表里三个 layer 的书写顺序是 `check_origin`、`security_headers`、`check_host`。一个请求进来，最先执行的是哪个？

A `check_origin`

它写在最前面，但它是最里层，离 handler 最近，最后执行。

B `security_headers`

它在中间。

C `check_host`

每一层 layer 都包住它上面的所有东西，最后加的在最外层，请求最先碰到它。



选择题

random_pin 为什么不直接写 $n \% 1_000_000$ ，而要先判断 n 小于一个上限？

A 为了更快

B 为了让每个口令出现的概率完全相同

C 为了不生成以 0 开头的口令



选择题

random_pin 为什么不直接写 $n \% 1_000_000$ ，而要先判断 n 小于一个上限？

A 为了更快

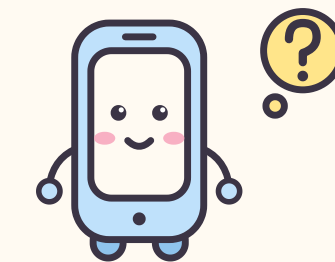
恰恰相反，偶尔要重抽一次，只会更慢一点点。

C 为了不生成以 0 开头的口令

以 0 开头的口令是允许的，格式化时会补零。

B 为了让每个口令出现的概率完全相同

32 位整数的个数不是一百万的整数倍，直接取余会让一部分口令多出现一次。只用整倍数以内的随机数，每个口令的概率就完全一样了。



选择题

改了 `web/index.html`，没有重新编译，直接运行旧的 `LanShare.exe`，浏览器看到的是哪个版本？

A 新的，服务器每次都读磁盘

B 旧的，网页在编译时就嵌进了 exe

C 打不开



选择题

改了 `web/index.html`，没有重新编译，直接运行旧的 `LanShare.exe`，浏览器看到的是哪个版本？

A 新的，服务器每次都读磁盘

`assets.rs` 用的是 `include_bytes`，运行时根本不读磁盘。

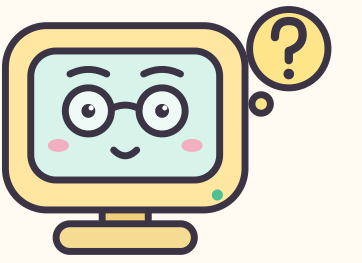
C 打不开

旧 `exe` 里有完整的旧网页，照样能打开。

B 旧的，网页在编译时就嵌进了 `exe`

`include_bytes` 在编译时把文件内容放进 `exe`。改了网页，必须重新编译。

想一想



check_host 已经挡住了域名，为什么还需要 check_origin? 两者分别防的是什么?



想一想

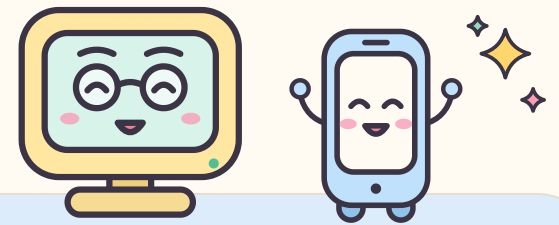
check_host 已经挡住了域名，为什么还需要 check_origin? 两者分别防的是什么?

check_host 防 DNS rebinding: 请求打着恶意域名进来，响应还会被交给恶意脚本。check_origin 防普通的跨站请求: 捣蛋鬼的网页直接请求 127.0.0.1, Host 是合法 IP, check_host 拦不住; 但浏览器会带上恶意网站的 Origin, 和 Host 对不上, 于是 403。



这一集写了什么

- ✓ async 函数返回 Future，tokio 运行时负责推动它们；main 里用 block_on 跑起来
- ✓ Config 是外面给的，AppState 是运行中共享的；Arc 共享一份，要改的字段放进 Mutex
- ✓ trait 写下约定，dyn 运行时再决定具体类型；Desktop 和 Logger 由外面插进来
- ✓ 三道关：check_host 防 DNS rebinding，security_headers 加 CSP，check_origin 挡跨站请求
- ✓ macro_rules! 生成六个 handler，include_bytes! 把网页编进 exe



下一集

全课完结，感谢收看